Azure DevOps
Complete Workflow Guide

From Clone to Production

End-to-End Development Operations Manual

✓ Branch Management
Complete workflow procedures

✓ PR & Review Process
Approval & conflict resolution

ISO/IEC 27001 Aligned | DevSecOps Framework

Version 1.0.0

Publication Date: February 09, 2026

Prepared by: Pargesoft DevOps Team

INTERNAL USE ONLY

Confidential - Pargesoft Proprietary Information

Table of Contents

Executive Summary

This comprehensive guide documents the complete Azure DevOps workflow at Pargesoft, from initial repository clone through production deployment. It provides step-by-step procedures for all development operations, ensuring consistency, quality, and compliance across all teams.

Document Scope:

• Complete end-to-end development workflow

• Repository cloning and branch management

• Work item integration and traceability

• Pull request creation and code review procedures

• Approval workflows and conflict resolution

• Build pipeline integration and validation

• Tag management and release procedures

• ISO/IEC 27001 and DevSecOps compliance

Target Audience:

Software developers, DevOps engineers, team leads, project managers, and QA engineers working with Azure DevOps at Pargesoft.

1. Getting Started - Azure DevOps Setup

Before beginning development work, ensure your Azure DevOps environment is properly configured.

1.1 Prerequisites

Requirement	Details	Installation
Azure DevOps Account	Pargesoft organization access	Contact IT Admin
Git Client	Version 2.30 or higher	https://git-scm.com
IDE/Editor	VS Code, Visual Studio, etc.	Per team standards
Permissions	Repository contributor access	Request via ticket

1.2 Git Configuration

```
# Configure your identity
git config --global user.name "Your Name"
git config --global user.email "yourname@pargesoft.com"

# Set default branch name
git config --global init.defaultBranch main

# Enable credential caching
git config --global credential.helper cache

# Set line ending handling
git config --global core.autocrlf true  # Windows
git config --global core.autocrlf input # Mac/Linux

# Verify configuration
git config --list
```

1.3 Azure DevOps Access Setup

Step 1: Navigate to Azure DevOps → Open browser to https://dev.azure.com/pargesoft Step 2: Authenticate → Sign in with your Pargesoft credentials → Enable MFA if prompted Step 3: Select Project → Choose your project from the organization → Verify access to Repos section Step 4: Generate Personal Access Token (PAT) → Click user icon (top right) → Security → Personal Access Tokens → New Token → Name: "Development Workstation" → Organization: Pargesoft → Expiration: 90 days (recommended) → Scopes: Code (Read, Write, Manage) → Create and securely store the token

■ **Security Note:** Never commit your PAT to version control. Store it securely in a password manager.

2. Clone Repository - Getting the Code

The first step in any development workflow is obtaining a local copy of the repository.

2.1 Obtaining Repository URL

1. Navigate to Azure DevOps → Your Project → Repos → Files 2. Click "Clone" button (top right) 3. Copy the HTTPS URL (recommended) or SSH URL Example: https://dev.azure.com/pargesoft/ProjectName/_git/RepositoryName

2.2 Clone Command Execution

```
# Open terminal/command prompt
# Navigate to your workspace directory
cd /path/to/workspace

# Clone the repository
git clone https://dev.azure.com/pargesoft/ProjectName/_git/RepositoryName

# Enter credentials when prompted
# Username: Your email
# Password: Your PAT (not your login password)

# Navigate into repository
cd RepositoryName

# Verify clone success
git status
git branch -a
```

2.3 Post-Clone Verification

✓ Repository folder created in workspace ✓ Hidden .git folder exists (contains version history) ✓ All files from main/default branch are present ✓ Git status shows "On branch main" or "On branch sandbox" ✓ No errors in terminal output

2.4 Troubleshooting Clone Issues

Error	Cause	Solution
Authentication failed	Invalid credentials	Verify PAT is correct and not expired

Permission denied	Insufficient access	Request repository access from admin
Repository not found	Incorrect URL	Verify URL from Azure DevOps clone button
SSL certificate problem	Network/proxy issue	Contact IT support for certificate

3. Branch Creation and Management

Pargesoft follows a structured branch hierarchy to ensure code quality and deployment control.

3.1 Branch Hierarchy Overview

Branch	Environment	Source	Target	Lifespan
main	Production	-	-	Permanent
release	Pre-Production	-	main	Permanent
sandbox	Development	-	release	Permanent
feature/*	Local	sandbox	sandbox	Temporary
bugfix/*	Local	release	release	Temporary
hotfix/*	Local	main	main	Temporary

3.2 Branch Naming Convention

```
Format: <type>/<workitem-id>-<short-description>

Examples:
  feature/1234-user-authentication
  feature/2567-payment-integration
  bugfix/3456-date-validation
  hotfix/9999-critical-security-patch

Rules:
  • Use lowercase letters
  • Separate words with hyphens
  • Always include work item ID
  • Keep description concise (3-5 words max)
  • No special characters except hyphens
```

3.3 Creating a Feature Branch

```
# Step 1: Ensure you're on sandbox branch
git checkout sandbox
git pull origin sandbox
```

```
# Step 2: Verify you have latest changes
git status

# Step 3: Create feature branch
git checkout -b feature/1234-sales-dashboard

# Step 4: Verify new branch creation
git branch
# Should show: * feature/1234-sales-dashboard

# Step 5: Push branch to remote (optional - recommended)
git push -u origin feature/1234-sales-dashboard
```

Best Practice: Create feature branches immediately after work item assignment to establish traceability from the start.

4. Work Item Workflow

Work items provide traceability between business requirements and code changes. All development work must be linked to a work item.

4.1 Work Item Types at Pargesoft

Type	Purpose	Created By	Branch Type
Epic	Large initiative spanning multiple sprints	Product Owner	N/A
Feature	Significant new functionality	Product Owner	feature/*
User Story	User-centric requirement	Product Owner	feature/*
Task	Technical implementation unit	Developer	feature/*
Bug	Defect in existing functionality	QA/Developer	bugfix/*

4.2 Work Item Assignment Process

Step 1: Review Backlog → Azure DevOps → Boards → Backlog → Sprint planning assigns work items to team members Step 2: Accept Assignment → Update work item state to "Active" → Add yourself as assigned to → Add any additional details or questions Step 3: Review Requirements → Read acceptance criteria → Review attached documents → Clarify any ambiguities with Product Owner Step 4: Create Branch → Note work item ID (e.g., 1234) → Create branch: feature/1234-description → Link branch to work item (automatic via naming) Step 5: Begin Development → Update work item with progress notes → Move to "In Progress" state

4.3 Work Item State Lifecycle

```
Work Item States:

New → Active → In Progress → Code Review → Testing → Done → Closed

State Transitions:
  New → Active: Work item assigned to developer
  Active → In Progress: Development started
  In Progress → Code Review: PR created
  Code Review → Testing: PR approved and merged
  Testing → Done: QA validation passed
  Done → Closed: Deployed to production
```

```
Rollback Transitions:
  Code Review → In Progress: Changes requested
  Testing → In Progress: Bugs found during QA
```

5. Development Cycle - Code, Commit, Push

This section covers the day-to-day development workflow from writing code to pushing changes.

5.1 Daily Development Workflow

```
# Start of day: Update your branch
git checkout feature/1234-sales-dashboard
git pull origin sandbox  # Get latest changes from sandbox

# Make code changes in your IDE
# ... implement feature ...

# Check what files changed
git status

# Review your changes
git diff

# Stage specific files
git add src/components/SalesDashboard.js
git add src/styles/dashboard.css

# OR stage all changes
git add .

# Commit with work item reference
git commit -m "[FEATURE] #1234 - Implemented sales dashboard main layout"

# Push to remote
git push origin feature/1234-sales-dashboard
```

5.2 Commit Message Standards

```
Format: [TYPE] #WORKITEM-ID - Description

Types:
  [FEATURE]  - New functionality
  [BUGFIX]   - Bug fix
  [HOTFIX]   - Critical production fix
  [REFACTOR] - Code improvement without functionality change
  [DOCS]     - Documentation updates
  [TEST]     - Test additions or modifications
  [STYLE]    - Code formatting, no logic change

Examples:
  ✓ [FEATURE] #1234 - Added user authentication module
  ✓ [BUGFIX] #5678 - Fixed date format validation error
  ✓ [REFACTOR] #2345 - Optimized database query performance
```

```
✓ [DOCS] #3456 - Updated API documentation

✗ "Fixed bug" - Missing type, work item, and details
✗ "WIP" - Not descriptive
✗ "asdfasdf" - Meaningless
```

Compliance Requirement: All commits must reference a work item ID for audit trail compliance (ISO/IEC 27001 A.8.15).

5.3 Multiple Commits Strategy

```
Approach 1: Logical Commits (Recommended)
# Break work into logical units
git add src/authentication/login.js
git commit -m "[FEATURE] #1234 - Implemented login form UI"

git add src/authentication/validation.js
git commit -m "[FEATURE] #1234 - Added input validation logic"

git add tests/authentication.test.js
git commit -m "[FEATURE] #1234 - Added unit tests for authentication"

Approach 2: Single Comprehensive Commit
# Complete all work, then commit
git add .
git commit -m "[FEATURE] #1234 - Implemented complete authentication module with tests"

Best Practice: Use logical commits for better code review and rollback capability
```

6. Pull Request (PR) Creation

Once your feature is complete, create a Pull Request to merge changes into the target branch.

6.1 Pre-PR Checklist

Before creating a PR, ensure: Code Quality: ■ Code compiles without errors ■ All tests pass locally ■ Code follows Pargesoft coding standards ■ No console.log or debugging code remains ■ No commented-out code blocks Documentation: ■ Code comments added for complex logic ■ README updated if needed ■ API documentation updated if applicable Testing: ■ Unit tests written for new functionality ■ Manual testing completed ■ Edge cases considered and tested Work Item: ■ Work item ID in branch name ■ Work item ID in commit messages ■ Work item status updated to "Code Review"

6.2 Creating PR in Azure DevOps

Step 1: Navigate to Pull Requests → Azure DevOps → Repos → Pull Requests → Click "New pull request" Step 2: Select Source and Target Branches → Source branch: feature/1234-sales-dashboard → Target branch: sandbox (for features) → Verify branch selection is correct Step 3: Fill PR Details → Title: [FEATURE] #1234 - Sales Dashboard Implementation → Description: Use template below → Add reviewers (minimum required per policy) → Link work items Step 4: Configure PR Options → ■ Delete source branch after merge (recommended) → ■ Complete work items after merge → Review auto-complete settings if available Step 5: Create PR → Click "Create" button → PR is now pending review

6.3 PR Description Template

```
## Summary
Brief description of changes made in this PR.

## Related Work Item
Fixes #1234

## Changes Made
- [ ] Implemented sales dashboard main component
- [ ] Added data fetching logic from API
- [ ] Created responsive CSS styling
- [ ] Added unit tests for dashboard component

## Testing Performed
- [ ] Unit tests: All passing (15 tests)
- [ ] Manual testing: Dashboard displays correctly
- [ ] Browser testing: Chrome, Firefox, Edge
- [ ] Responsive testing: Mobile, tablet, desktop

## Screenshots/Demo
```

```
[Attach screenshots or GIF of functionality]

## Breaking Changes
None / [List any breaking changes]

## Security Considerations
- API authentication validated
- Input sanitization implemented
- No sensitive data exposed

## Additional Notes
[Any additional context for reviewers]
```

6.4 Understanding PR Policies

Policy	sandbox	release	main
Minimum reviewers	1	2	2
Build validation	Required	Required	Required
Work item linking	Optional	Required	Required
Comment resolution	Required	Required	Required

7. Code Review Process

Code review is a critical quality gate ensuring code meets Pargesoft standards.

7.1 Reviewer Responsibilities

As a reviewer, you must evaluate: Code Quality: • Code is readable and maintainable • Follows Pargesoft coding conventions • No code smells or anti-patterns • Proper error handling implemented • Logging added for critical operations Functionality: • Code implements requirements from work item • Edge cases are handled • No obvious bugs or logical errors • Performance considerations addressed Security: • No hardcoded credentials or secrets • Input validation implemented • SQL injection prevention • XSS protection in place • Authentication/authorization enforced Testing: • Unit tests provide adequate coverage • Tests are meaningful and not trivial • Test cases cover edge scenarios • Integration tests added if needed Documentation: • Complex logic is commented • Public APIs documented • README updated if needed

7.2 Performing Code Review

Step 1: Access PR → Navigate to Azure DevOps → Pull Requests → Click on assigned PR Step 2: Review Overview → Read PR title and description → Verify work item linkage → Check source/target branches are correct Step 3: Review Code Changes → Click "Files" tab → Review each changed file → Look for issues in categories above Step 4: Add Comments → Inline comments: Click line number, add comment → General comments: Use "Overview" tab → Be specific and constructive → Suggest improvements, not just criticism Step 5: Request Changes or Approve → If issues found: Click "Request changes" button → If acceptable: Click "Approve" button → Add summary comment explaining decision

7.3 Effective Review Comments

```
Good Comments (Constructive):
✓ "Consider extracting this logic into a separate function for reusability"
✓ "This might throw a NullPointerException if data is undefined. Add null check?"
✓ "Great implementation! Minor suggestion: use const instead of let here for immutability"
✓ "Missing unit test for the error handling branch. Can you add one?"

Poor Comments (Avoid):
✗ "This is wrong" - Not specific, not helpful
✗ "Why did you do it this way?" - Sounds accusatory
✗ "I would never write code like this" - Personal attack
✗ "Fix this" - No explanation of what or why

Tone Guidelines:
• Be respectful and professional
• Assume positive intent
• Explain the "why" behind suggestions
• Praise good work when you see it
• Use questions instead of commands
```

8. Approval Workflow

PR approval is a formal process ensuring code meets quality and compliance standards.

8.1 Approval Requirements by Branch

Target Branch	Required Approvers	Who Can Approve	Auto-Complete
sandbox	1	Any team member	Allowed
release	2	Tech lead or senior dev	Allowed with policy
main	2	Tech lead + DevOps lead	Restricted

8.2 As PR Author - Responding to Reviews

When Changes Are Requested: Step 1: Review Comments → Read all reviewer comments carefully → Acknowledge receipt: Reply "Understood" or "Working on it" Step 2: Address Comments → Make code changes based on feedback → Commit changes with descriptive message → Push to feature branch # Example: Addressing review comments git add src/components/Dashboard.js git commit -m "[FEATURE] #1234 - Addressed review comments: Added null checks" git push origin feature/1234-sales-dashboard Step 3: Respond to Comments → Reply to each comment in Azure DevOps → Mark comments as resolved if fixed → Explain if you disagree (politely) Step 4: Request Re-review → Click "Request re-review" button → Or add comment: "@reviewer Ready for re-review" Step 5: Address Additional Rounds → Repeat until all reviewers approve → Maintain professional communication throughout

8.3 Handling Disagreements

If you disagree with a review comment: Approach 1: Polite Discussion "I understand your concern about X. However, I chose this approach because Y. What do you think about this reasoning?" Approach 2: Request Clarification "Can you explain more about why you prefer approach X over Y? I want to understand the trade-offs better." Approach 3: Escalate Appropriately If consensus cannot be reached: → Request tech lead input → Schedule brief discussion call → Document decision rationale Remember: • Code review is collaborative, not adversarial • Goal is better code, not winning arguments • Sometimes there are multiple valid approaches • When in doubt, follow team/company standards

9. Merge Conflict Resolution

Merge conflicts occur when changes in different branches affect the same code. Resolving conflicts requires careful attention to preserve both sets of changes correctly.

9.1 Understanding Merge Conflicts

A merge conflict happens when: • You and another developer modify the same file • Git cannot automatically determine which changes to keep • Manual intervention is required to resolve Common Scenarios: 1. Two developers edit the same function 2. One developer deletes a file another modified 3. Conflicting changes to configuration files 4. Parallel changes to the same CSS rules

9.2 Detecting Conflicts in Azure DevOps

Visual Indicators:

In your PR, you will see: • ■ Warning icon next to "Merge" button • Message: "Merge conflicts must be resolved" • "Resolve conflicts" button becomes available • List of conflicting files shown Azure DevOps provides two resolution methods: 1. Web-based editor (simple conflicts) 2. Local resolution (complex conflicts - recommended)

9.3 Resolving Conflicts Locally (Recommended)

```
# Step 1: Ensure you're on your feature branch
git checkout feature/1234-sales-dashboard

# Step 2: Fetch latest changes from remote
git fetch origin

# Step 3: Attempt merge from target branch
git merge origin/sandbox

# Output will show conflicts:
# Auto-merging src/components/Dashboard.js
# CONFLICT (content): Merge conflict in src/components/Dashboard.js
# Automatic merge failed; fix conflicts and then commit the result.

# Step 4: Check which files have conflicts
git status

# Output shows:
# Unmerged paths:
#   both modified:   src/components/Dashboard.js

# Step 5: Open conflicting file in your IDE
# You'll see conflict markers:
```

```
<<<<<<< HEAD
// Your changes
const dashboard = new Dashboard(config);
=======
// Their changes
const dashboard = createDashboard(settings);
>>>>>>> origin/sandbox

# Step 6: Resolve conflict manually
# Choose one approach or combine both:
# Option A: Keep your changes (remove their changes)
# Option B: Keep their changes (remove your changes)
# Option C: Combine both changes logically

# After resolution, file should look like:
const dashboard = new Dashboard(settings);

# Step 7: Stage resolved file
git add src/components/Dashboard.js

# Step 8: Continue merge
git commit -m "[FEATURE] #1234 - Resolved merge conflicts with sandbox"

# Step 9: Push resolved changes
git push origin feature/1234-sales-dashboard

# Step 10: PR automatically updates in Azure DevOps
```

9.4 Conflict Resolution Strategies

Scenario	Strategy	Action
Simple text change	Keep both if compatible	Manually combine changes
Function rename	Coordinate with other dev	Choose consistent name
Deleted vs modified file	Verify requirement	Keep if still needed
Configuration conflict	Preserve both configs	Merge configuration values

■ **Critical:** After resolving conflicts, thoroughly test your code. Conflicts can introduce subtle bugs if not resolved correctly.

10. Build Pipeline and Validation

Automated build pipelines validate code quality, run tests, and ensure deployability before merging to protected branches.

10.1 Build Pipeline Stages

Standard Pargesoft Build Pipeline: Stage 1: Source Code Compilation → Code is compiled/transpiled → Syntax errors detected → Dependencies resolved Stage 2: Unit Tests → All unit tests executed → Minimum 80% code coverage required → Test results published Stage 3: Static Code Analysis → SonarQube analysis → Code quality metrics evaluated → Code smells identified Stage 4: Security Scanning → SAST (Static Application Security Testing) → Dependency vulnerability check (OWASP) → Secret scanning (no credentials in code) Stage 5: Build Artifacts → Deployable artifacts created → Docker images built (if applicable) → Artifacts uploaded to artifact repository Stage 6: Deployment (if applicable) → Deploy to development environment → Smoke tests executed → Health checks performed

10.2 When Builds Execute

Event	Validation Build	Full Build	Deployment
PR to sandbox	✓	✗	✗
Merge to sandbox	✗	✗	✗
Merge to release	✗	✓	Pre-Production
Merge to main	✗	✓	Production

10.3 Handling Build Failures

When a build fails on your PR: Step 1: Identify Failure Point → Navigate to PR in Azure DevOps → Click "Build" or "Checks" tab → Click on failed build to see logs Step 2: Analyze Build Logs → Scroll to red/error messages → Identify specific failure (compilation, test, security, etc.) → Note exact error message and stack trace Step 3: Reproduce Locally # Run the same checks locally npm test # For JavaScript/TypeScript dotnet test # For .NET mvn test # For Java ./gradlew test # For Gradle Step 4: Fix Issue → Correct the code based on error → Verify fix works locally → Commit and push fix git add . git commit -m "[FEATURE] #1234 - Fixed unit test failure in Dashboard component" git push origin feature/1234-sales-dashboard Step 5: Verify Build Passes → Build automatically re-runs on push → Monitor build status in Azure DevOps → Proceed with merge only after green build

10.4 Common Build Failures

Failure Type	Common Causes	Resolution
Compilation Error	Syntax error, missing import	Fix code syntax, add imports
Test Failure	Broken test, logic error	Fix test or implementation
Coverage Drop	New code not tested	Add unit tests
Security Scan	Known vulnerability	Update dependency version
Code Quality	Code smells, duplication	Refactor per SonarQube

11. Merge to Target Branch

After all approvals and validations pass, the PR can be merged to the target branch.

11.1 Pre-Merge Verification

Before completing merge, verify: ✓ All required approvals received ✓ All review comments resolved ✓ Build pipeline passed (green check) ✓ No merge conflicts present ✓ Work item linked correctly ✓ Branch is up to date with target

11.2 Merge Strategies at Pargesoft

```
Pargesoft supports two merge strategies:

1. Merge Commit (Default, Recommended)
   • Creates a merge commit
   • Preserves complete commit history
   • Shows branch structure in history
   • Use for: Most PRs, especially multi-commit features

   Result: All commits preserved + merge commit

2. Squash Merge
   • Combines all commits into single commit
   • Cleaner linear history
   • Loses individual commit details
   • Use for: Small fixes, work-in-progress cleanup

   Result: Single commit with combined changes

NOT ALLOWED:
✗ Rebase and fast-forward (disabled by policy)
✗ Force push to protected branches
```

11.3 Completing the Merge

Step 1: Navigate to PR → Azure DevOps → Pull Requests → Your PR Step 2: Verify Ready to Merge → Green "Complete" button is active → All policies satisfied (shown with checks) Step 3: Click "Complete" Button → Review merge summary dialog Step 4: Configure Merge Options → Merge type: Merge commit (or Squash if appropriate) → ■ Delete feature branch after merge (recommended) → ■ Complete associated work items → ■ Customize merge commit message (optional) Step 5: Complete Merge → Click "Complete merge" button → Confirmation appears → Branch is merged to target Step 6: Verify Merge Success → PR status changes to "Completed" → Feature branch deleted (if selected) → Work item status updated → Build pipeline triggered (if configured)

ISO/IEC 27001 Compliance: Merge operations are logged for audit trail. Work item linkage ensures traceability of all changes to business requirements.

12. Back-Merge Operations

Back-merge ensures changes propagate from higher environments back to lower environments, preventing branch divergence.

12.1 Automated Back-Merge Flows

```
Pargesoft Automated Back-Merge Rules:

Scenario 1: After sandbox → release merge
→ Automatic PR created: release → sandbox
→ Purpose: Sync any release-specific changes back

Scenario 2: After bugfix → release merge
→ Automatic PR created: release → sandbox
→ Purpose: Ensure bug fixes reach development

Scenario 3: After release → main merge
→ Automatic PR #1: main → release
→ Automatic PR #2: release → sandbox
→ Purpose: Cascade production changes to all branches

Scenario 4: After hotfix → main merge
→ Automatic PR #1: main → release
→ Automatic PR #2: release → sandbox
→ Purpose: Critical fixes propagate to all environments

Note: Back-merge PRs are auto-created but may require manual merge
```

12.2 Resolving Back-Merge Conflicts

If automatic back-merge PR has conflicts: Step 1: You'll be notified via email/Azure DevOps Step 2: Navigate to the back-merge PR Step 3: Follow conflict resolution process (Section 9) Step 4: Resolve conflicts locally: # Example: Resolving release → sandbox back-merge git checkout sandbox git pull origin sandbox git merge origin/release # Fix conflicts in IDE git add . git commit -m "Resolved back-merge conflicts" git push origin sandbox Step 5: Back-merge PR automatically updates Step 6: Approve and complete merge Best Practice: Regularly sync your feature branches with sandbox to minimize back-merge conflicts

13. Tag and Release Management

Tags mark specific points in history, typically for production releases. Pargesoft follows Semantic Versioning (SemVer) for release tagging.

13.1 Semantic Versioning at Pargesoft

```
Version Format: vMAJOR.MINOR.PATCH

MAJOR version (v2.0.0):
• Incompatible API changes
• Breaking changes to existing functionality
• Requires updates to dependent systems
• Example: v1.5.3 → v2.0.0

MINOR version (v1.5.0):
• New features added
• Backward-compatible functionality
• No breaking changes
• Example: v1.4.2 → v1.5.0

PATCH version (v1.5.3):
• Bug fixes only
• Backward-compatible fixes
• No new features
• Example: v1.5.2 → v1.5.3

Pre-release versions:
• v1.5.0-alpha.1 (early alpha)
• v1.5.0-beta.2 (beta testing)
• v1.5.0-rc.1 (release candidate)

Build metadata:
• v1.5.0+20250209 (build date)
• v1.5.0+001 (build number)
```

13.2 Creating Release Tags

```
Method 1: Via Terminal (Recommended)

# Step 1: Ensure on main branch
git checkout main
git pull origin main

# Step 2: Create annotated tag
git tag -a v1.5.0 -m "Release 1.5.0 - Sales Dashboard Feature"

# Step 3: Push tag to remote
git push origin v1.5.0
```

```
# Step 4: Verify tag created
git tag -l "v1.*"

Method 2: Via Azure DevOps UI

Step 1: Navigate to Repos → Tags
Step 2: Click "New tag"
Step 3: Enter tag details:
   • Name: v1.5.0
   • Based on: main
   • Description: Release notes
Step 4: Click "Create"
```

13.3 Release Notes Template

```
# Release v1.5.0 - Sales Dashboard

## Release Date
February 9, 2025

## New Features
- [#1234] Sales Dashboard with real-time analytics
- [#2345] Export to PDF functionality
- [#3456] Customizable date range filtering

## Bug Fixes
- [#5678] Fixed date format validation error
- [#6789] Resolved chart rendering on mobile devices

## Improvements
- [#7890] Optimized database query performance
- [#8901] Enhanced error handling in API calls

## Breaking Changes
None

## Deployment Notes
- Requires database migration: migrations/2025-02-09-dashboard.sql
- Update configuration: Add DASHBOARD_API_KEY to environment

## Known Issues
- Dashboard export may timeout for very large datasets
- Workaround: Filter data to smaller range before export

## Contributors
- Jane Doe
- John Smith
- Alice Johnson
```

14. Troubleshooting Guide

Common issues and solutions for Azure DevOps workflow problems.

14.1 Common Issues and Solutions

Issue	Symptom	Solution
Authentication Failed	Cannot push/pull	Regenerate PAT, verify credentials
Branch Policy Violation	PR rejected	Follow correct branch flow (Sec 3)
Build Failure	Red X on PR	Check logs, fix errors locally (Sec 10)
Merge Conflict	Cannot merge PR	Resolve conflicts locally (Sec 9)
Missing Approvals	Cannot complete PR	Request reviews from required approvers
Work Item Not Linked	Policy blocks PR	Link work item in PR description
Outdated Branch	Conflicts on merge	Update branch: git pull origin sandbox

14.2 Getting Help

Pargesoft Support Resources: Level 1: Team Resources • Ask teammate for peer support • Review this documentation • Check Azure DevOps documentation Level 2: Team Lead • Complex technical issues • Process clarification • Policy exceptions Level 3: DevOps Team • Build pipeline issues • Branch policy problems • Azure DevOps configuration Level 4: IT Support • Access and permissions • Account issues • Network/connectivity problems Contact Methods: • Teams: #devops-support channel • Email: devops@pargesoft.com • Ticket: IT Service Portal • Urgent: Call DevOps on-call (emergencies only)

15. Quick Reference Cheat Sheet

15.1 Essential Git Commands

Task	Command
Clone repository	git clone <url>
Check status	git status
Create branch	git checkout -b feature/123-name
Switch branch	git checkout sandbox
Pull latest	git pull origin sandbox
Stage changes	git add .
Commit	git commit -m "[TYPE] #123 - Description"
Push branch	git push origin feature/123-name
View history	git log --oneline --graph
Discard changes	git checkout -- <file>

15.2 Standard Feature Workflow Summary

1. Get work item assignment 2. Clone repository (if first time) 3. Create feature branch from sandbox 4. Develop and commit changes 5. Push branch to remote 6. Create PR to sandbox 7. Address review comments 8. Get approval(s) 9. Resolve any conflicts 10. Ensure build passes 11. Complete merge 12. Verify work item closed 13. Delete feature branch (if not auto-deleted)

15.3 Branch Flow Reminder

```
Normal Development:
feature/* → sandbox → release → main

Bug Fix in Release:
bugfix/* → release → main

Critical Production Fix:
```

```
hotfix/* → main
(Then automatic back-merge to release and sandbox)

NEVER:
✗ feature/* → release (must go through sandbox)
✗ feature/* → main (must go through sandbox → release)
✗ sandbox → main (must go through release)
```

Appendix A: ISO/IEC 27001 Compliance

Pargesoft's Azure DevOps workflow implements controls aligned with ISO/IEC 27001:2022 information security management requirements.

Control ID	Control Name	Implementation
A.5.2	Segregation of duties	Separate approvers for prod deployments
A.5.3	Access control	Branch policies enforce min reviewers
A.8.9	Configuration mgmt	Git version control, tag management
A.8.15	Audit logging	Work item linkage, commit traceability
A.8.25	Secure development	Code review, security scanning
A.8.32	Change management	Mandatory PR workflow, approvals

Audit Trail: Every code change is traceable through work items, commits, PR reviews, and approvals, providing complete accountability.

Appendix B: DevSecOps Integration

Security is integrated throughout the development workflow, implementing shift-left security principles.

Stage	Security Control	Tool
Code Commit	Pre-commit hooks	Git hooks
PR Creation	Static analysis	SonarQube
Build	SAST scanning	Checkmarx
Build	Dependency check	OWASP Dependency-Check
Build	Secret scanning	GitGuardian
Deployment	DAST scanning	OWASP ZAP
Runtime	Monitoring	SIEM integration

Security-First Culture: All developers are responsible for security. Security issues found in pipelines must be resolved before merge approval.

Document Control

Version	Date	Author	Changes
1.0.0	2025-02-09	DevOps Team	Initial complete workflow documentation

End of Document - Pargesoft Proprietary Information